深度学习笔记

Attention Is All You Need

Attention Is All You Need

目录

学习笔记: Attention Is All You Need

1. 研究背景与问题定义

1.1 领域背景：序列转导模型的发展

在自然语言处理和序列转导任务（如机器翻译）领域，**循环神经网络（RNN）**及其变体，包括**长短期记忆网络（LSTM）**和**门控循环单元（GRU）**，长期以来占据着主导地位。这些模型采用**编码器-解码器（Encoder-Decoder）**架构：

- **编码器**：将输入序列 $(x_1, ..., x_n)$ 映射到连续表示 $z = (z_1, ..., z_n)$
- **解码器**：基于 z ，逐个生成输出序列 $(y_1, ..., y_m)$

1.2 之前存在的核心问题

1.2.1 顺序计算的固有瓶颈 传统 RNN 模型的计算沿着输入和输出序列的**符号位置**因式分解：

$$h_t = f(h_{t-1}, x_t)$$

其中 h_t 是位置 t 的隐藏状态，依赖于前一隐藏状态 h_{t-1} 和当前位置的输入 x_t。这种**固有的顺序性质**导致了两个严重问题：

1. **并行化受限**：在训练样本内部无法进行并行计算
2. **内存限制**：当序列较长时，跨样本的批处理（batching）受到限制

虽然后续工作通过**因子分解技巧**和**条件计算**改善了计算效率，但**顺序计算的基本约束仍未解决**。

1.2.2 长距离依赖的学习困难 在 RNN 中，两个位置之间的路径长度与它们之间的距离成正比 $O(n)$。这使得模型在**学习长距离依赖关系**时变得困难，因为梯度在反向传播过程中可能会**指数级衰减或爆炸**。

1.2.3 卷积方法的局限性 为解决 RNN 的并行化问题，一些工作（如 **ByteNet**、**ConvS2S**）使用**卷积神经网络（CNN）**作为基础构建块。然而：

- **ConvS2S**：关联两个任意位置所需的运算数随距离**线性增长**
- **ByteNet**：随距离**对数增长**

尽管优于 RNN，但仍未能达到**常数时间复杂度**的理想目标。

1.3 本文要解决的核心问题

本文（Transformer）旨在提出一个**完全基于注意力机制**的模型架构，彻底**摒弃循环和卷积操作**，从而：

1. **实现最大程度的并行化**
2. **缩短任意位置间的依赖路径长度**至常数级
3. **显著减少训练时间**
4. **在翻译质量上达到或超越现有最佳模型**

1.4 问题的重要性

- 机器翻译是 NLP 领域的核心任务，性能提升具有广泛的应用价值
- 序列转导是许多 NLP 任务的基础范式
- 训练效率的提升使得更大规模模型的训练成为可能
- 该架构的通用性使其可扩展到其他模态（音频、视频等）

---

2. 相关工作梳理

2.1 早期序列转导模型

模型	年份	核心特点	主要贡献
Seq2Seq (Sutskever et al.)	2014	RNN 编码器-解码器	首次将序列到序列学习用于机器翻译
Bahdanau Attention	2014	加入注意力机制的 Seq2Seq	允许解码器关注输入的相关部分
GNMT (Google)	2016	大规模 NMT 系统	工程化的端到端翻译系统

2.2 注意力机制的演进

注意力机制允许建模输入或输出序列中不同位置之间的依赖关系，而不受距离限制。在此之前，注意力机制通常**与 RNN 结合使用**：

- **Bahdanau 注意力**：加性注意力，通过前馈网络计算兼容性
- **Luong 注意力**：乘性（点积）注意力，计算更高效

2.3 降低顺序计算的努力

方法	采用的技术	路径长度	局限性
Extended Neural GPU	神经网络 + 卷积	-	计算量大
ByteNet	扩张卷积	$O(\log n)$	仍有距离相关复杂度
ConvS2S	卷积 + 注意力	$O(n)$	无法完全并行

2.4 本文与相关工作的关键区别

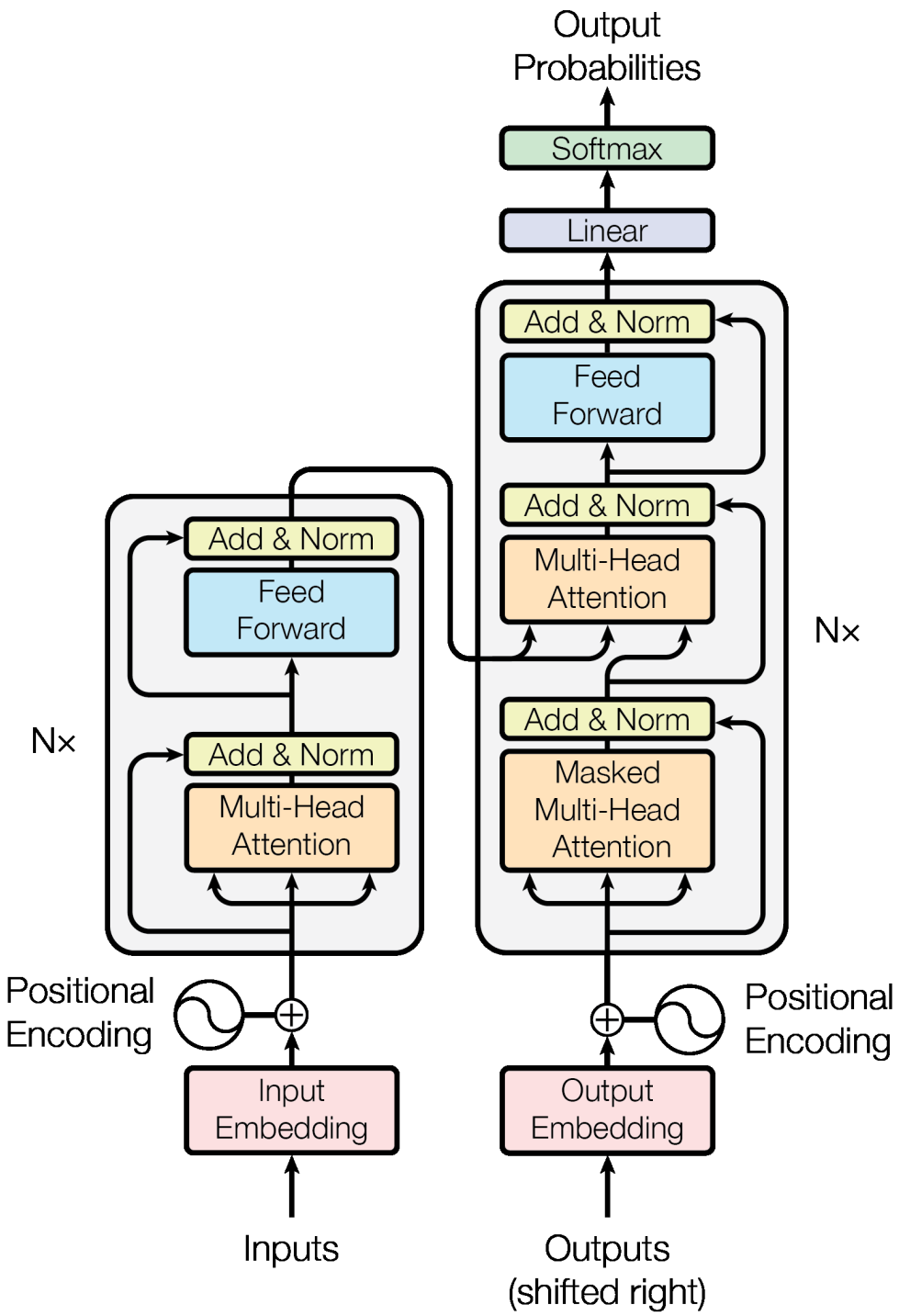
特征	传统 RNN	卷积方法	**Transformer（本文）**
计算方式	顺序	可并行	**完全可并行**
最大路径长度	$O(n)$	$O(n)$ 或 $O(\log n)$	**%%INLINEMATH14%%**
核心机制	循环	卷积	**自注意力**
依赖类型	必须配合注意力	需堆叠多层	**纯注意力**

2.5 自注意力的先驱工作

自注意力（Self-Attention），也称为**内部注意力**，是一种关联同一序列中不同位置以计算序列表示的注意力机制。之前的工作已将自注意力用于：

- 阅读理解
- 抽象摘要生成
- 文本蕴含
- 学习任务无关的句子表示
- 端到端记忆网络

然而，据本文所述，**Transformer 是第一个完全依赖自注意力、不使用任何序列对齐 RNN 或卷积的转导模型。**


Output
Probabilities
Softmax
Linear
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Add & Norm
Masked
Multi-Head
Attention
Nx
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Nx
Positional
Encoding
Positional
Encoding
Input
Embedding
Output
Embedding
Inputs
Outputs
(shifted right)

3. 核心方法详解

3.1 整体架构概述

如论文中的**图 1** 所示，Transformer 采用经典的**编码器-解码器结构**，但完全使用**堆叠的自注意力**和**逐位置的全连接层**。

3.1.1 编码器（Encoder）

- 由 $\mathbf{N = 6}$ 个相同层的堆叠组成
- 每层包含两个子层：

1. **多头自注意力机制**（Multi-Head Self-Attention）
2. **逐位置前馈网络**（Position-wise Feed-Forward Network）

- 采用**残差连接**（Residual Connection）：$\text{LayerNorm}(x + \text{Sublayer}(x))$
- 所有子层输出维度：**%%INLINEMATH16%%**

3.1.2 解码器（Decoder）

- 同样由 $\mathbf{N = 6}$ 个相同层的堆叠组成
- 每层包含三个子层：

1. **掩码多头自注意力**（Masked Multi-Head Attention）
2. **编码器-解码器注意力**（Encoder-Decoder Attention）
3. **逐位置前馈网络**

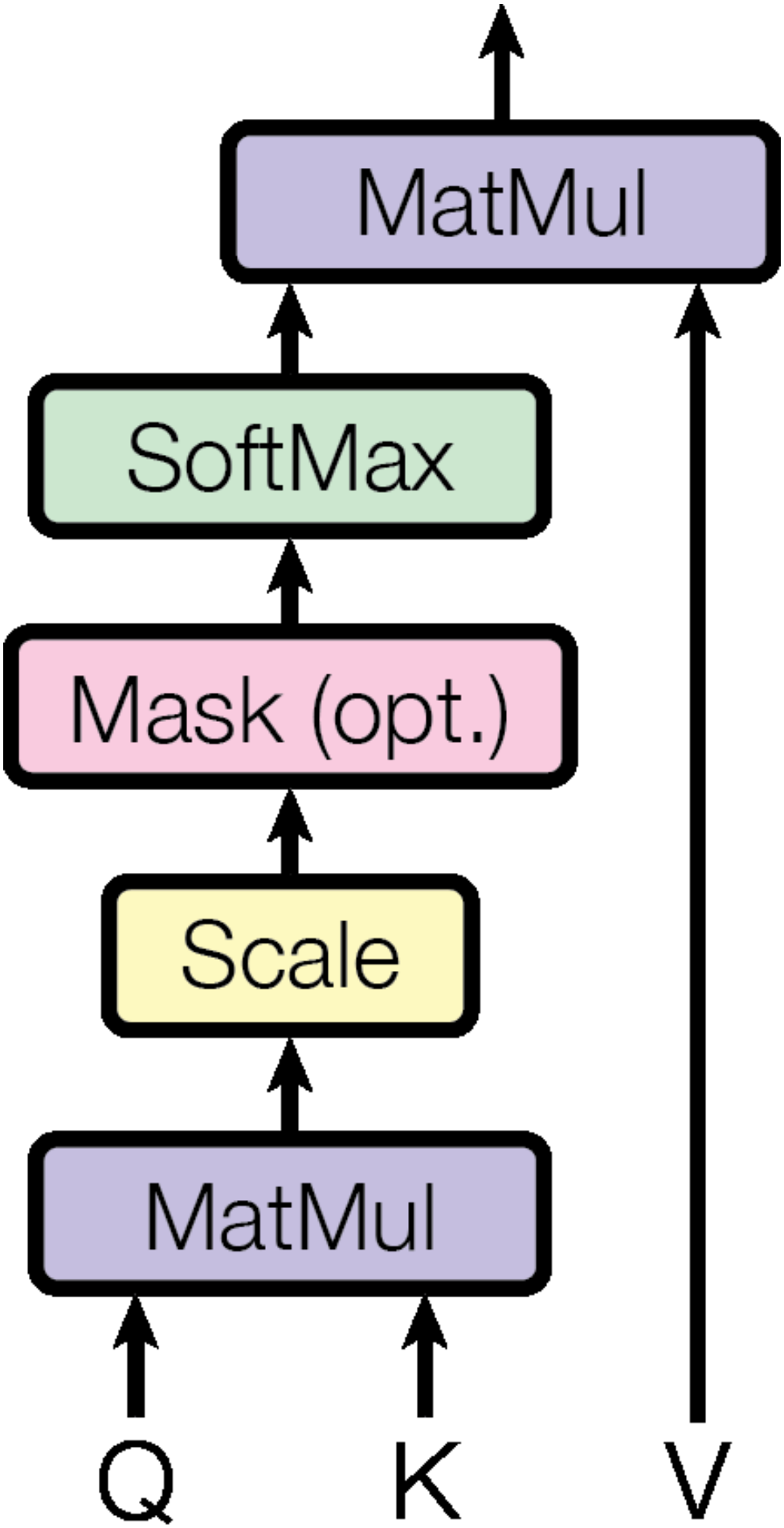
- 同样使用残差连接和层归一化
- **关键机制**：防止位置 i 关注后续位置（保持自回归特性）

3.2 注意力机制详解

3.2.1 缩放点积注意力（Scaled Dot-Product Attention） 这是 Transformer 核心的注意力函数。

输入：

- 查询（Query）：$\mathbf{Q} \in \mathbb{R}^{n \times d_k}$
- 键（Key）：$\mathbf{K} \in \mathbb{R}^{m \times d_k}$
- 值（Value）：$\mathbf{V} \in \mathbb{R}^{m \times d_v}$


MatMul
SoftMax
Mask (opt.)
Scale
MatMul
Q
K
V

核心公式:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

工作流程（参见论文**图 2 左**）:

1. 计算查询与所有键的点积：$\mathbf{Q}\mathbf{K}^T$
2. 缩放因子 $\frac{1}{\sqrt{d_k}}$ 防止点积过大
3. 应用 softmax 获取注意力权重
4. 加权求和得到输出

为什么需要缩放?
假设 q 和 k 的分量是均值为 0、方差为 1 的独立随机变量，则点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的:

- 均值为 0
- **方差为%%INLINEMATH26%%**

当 d_k 较大时，点积的量级会显著增大，将 softmax 函数推向**梯度极小的区域**。缩放因子 $\frac{1}{\sqrt{d_k}}$ 将方差归一化回 1。
与加性注意力的比较:

- 加性注意力使用前馈网络计算兼容性
- 点积注意力实现更简单、速度更快、空间效率更高
- 两者理论复杂度相似，但点积注意力可通过高度优化的矩阵乘法实现

3.2.2 多头注意力（Multi-Head Attention） 设计动机:

- 不使用单一的高维注意力，而是将查询、键、值**线性投影**到多个低维子空间
- 允许模型在**不同的表示子空间**中同时关注来自不同位置的信息
- 单头注意力的平均操作会抑制这种多样性

核心公式:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, ..., \text{head}_h)\mathbf{W}^O$$

其中:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

投影矩阵维度:

- $\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$

- $\mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$
- $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$

本文的具体设置：

- 头数 $h = 8$
- 每个头的维度 $d_k = d_v = d_{\text{model}}/h = 64$
- 由于每头维度减小，总计算成本与全维度单头注意力相当

多头注意力的优势：

- **并行计算**：各头可同时处理
- **参数效率**：总维度不变，但表达能力增强
- **多样性**：不同头可学习关注不同类型的依赖关系

3.2.3 Transformer 中注意力的三种应用

应用场景	查询来源	键/值来源	作用
编码器自注意力	编码器前一层的输出	编码器前一层的输出	允许编码器关注输入序列的所有位置
解码器自注意力	解码器前一层的输出	解码器前一层的输出	允许解码器关注当前位置及之前的位置
编码器-解码器注意力	解码器的上一子层	编码器的最终输出	允许解码器关注输入序列的相关部分

解码器中的掩码机制：

为保持自回归特性，解码器的自注意力层需要**防止向左信息流动**。通过将 softmax 输入中对应非法连接的位置设置为 $-\infty$ 来实现掩码：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

其中 $\mathbf{M}$ 是掩码矩阵，非法位置为 $-\infty$，合法位置为 0。

3.3 逐位置前馈网络（Position-wise Feed-Forward Networks）

除注意力子层外，编码器和解码器的每层还包含一个**全连接前馈网络**，对每个位置独立应用相同的变换：

$$\text{FFN}(x) = \max(0, x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

或等价地：

$$\text{FFN}(x) = \text{ReLU}(x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

参数配置：

- 输入/输出维度：$d_{\text{model}} = 512$
- 内部层维度：$d_{ff} = 2048$
- 可视为两个核大小为 1 的卷积

作用：

- 为每个位置提供非线性变换能力
- 增加模型的表达能力
- 计算可完全并行

3.4 位置编码（Positional Encoding）

由于 Transformer **不含循环和卷积**，无法自然地利用序列顺序信息。因此，必须注入关于**令牌相对或绝对位置**的信息。

3.4.1 编码方案 本文采用**正弦和余弦函数**的固定位置编码：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

其中：

- pos：位置（0, 1, 2, ...）
- i：维度索引（0, 1, 2, ..., $d_{\text{model}}/2$）

3.4.2 设计原理 对于任意固定偏移量 k，位置编码 PE_{pos+k} 可以表示为 PE_{pos} 的**线性函数**：

$$PE_{pos+k} = \mathbf{M} \cdot PE_{pos}$$

其中 $\mathbf{M}$ 是一个旋转矩阵。这使得模型能够**轻松学习关注相对位置**。

3.4.3 与可学习位置编码的比较 作者也实验了可学习的位置嵌入，发现两种方法产生了**几乎相同的结果**。选择正弦版本的原因：

- 可外推到**比训练时更长的序列**
- 无需学习额外参数

3.5 完整算法流程

编码器前向传播（单层）:

```
Input: x (batch_size, seq_len, d_model)
1. x' = LayerNorm(x + MultiHeadSelfAttention(x))
2. output = LayerNorm(x' + FFN(x'))
```

解码器前向传播（单层）:

```
Input: x (decoder input), encoder_output
1. x' = LayerNorm(x + MaskedMultiHeadSelfAttention(x))
2. x'' = LayerNorm(x' + MultiHeadAttention(x', encoder_output, encoder_output))
3. output = LayerNorm(x'' + FFN(x''))
```

3.6 与其他模型的复杂度比较

[TABLE:table1 - Maximum path lengths, per-layer complexity]
根据论文中的**表 1**，各层类型的复杂度比较:

层类型	每层复杂度	顺序操作	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环层	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积层	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
限制自注意力	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

关键洞察:

- 当序列长度 n **小于**表示维度 d 时（通常如此，如词片表示），自注意力比循环层更快
- 自注意力层的**最大路径长度为常数%%INLINEMATH61%%**，优于所有其他方法

3.7 可视化分析

论文中的**图 3** 展示了一个典型的注意力分布示例，显示编码器自注意力在第 5 层的注意力头能够捕捉长距离依赖——动词"making" 与"more difficult" 之间的语义关联。

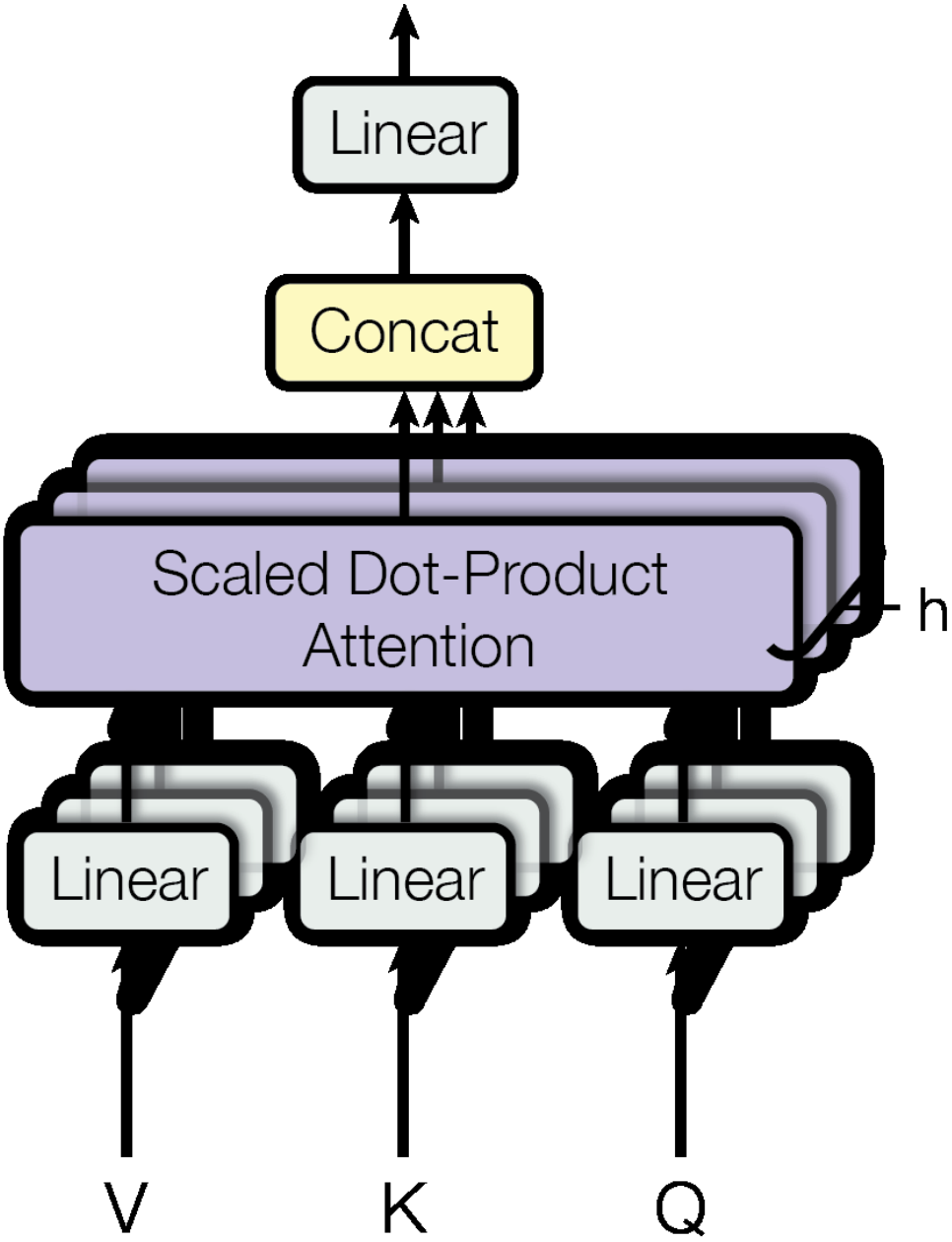
---

4. 实验设计与结果分析

4.1 数据集

4.1.1 机器翻译任务

数据集	语言对	句子对数量	词汇表	词片大小
WMT 2014	英德	~450 万	共享 37000	BPE
WMT 2014	英法	~3600 万	32000	词片


Linear
Concat
Scaled Dot-Product
Attention
h
Linear
Linear
Linear
V
K
Q

4.1.2 训练批次

- 按近似序列长度分批
- 每批次约包含 **25000 个源令牌**和 **25000 个目标令牌**

4.2 评估指标

- **BLEU**：机器翻译的标准评估指标，衡量生成翻译与参考翻译的 n-gram 重叠度
- **Perplexity**：困惑度，衡量语言模型的质量

4.3 训练配置

4.3.1 硬件

- **8 块 NVIDIA P100 GPU**

4.3.2 训练时间

模型	每步时间	总步数	总训练时间
Base 模型	~0.4 秒	100,000 步	~12 小时
Big 模型	~1.0 秒	300,000 步	~3.5 天

4.3.3 优化器 使用 **Adam 优化器**，配合学习率预热策略：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min\left(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}\right)$$

- $\beta_1 = 0.9$
- $\beta_2 = 0.98$
- $\epsilon = 10^{-9}$
- $warmup_steps = 4000$

该策略在**前 4000 步线性增加学习率**，之后按步数的平方根倒数递减。

4.3.4 正则化

1. **残差 Dropout**：

- 对每个子层输出的 dropout 率 $P_{drop} = 0.1$
- 编码器/解码器栈中嵌入与位置编码和的 dropout

1. **标签平滑**：

- $\epsilon_{ls} = 0.1$
- 虽增加困惑度，但提升准确率和 BLEU 分数

4.3.5 推理策略

- **束搜索**（Beam Search）：beam size = 4
- **长度惩罚**：$\alpha = 0.6$
- 最大输出长度：输入长度 + 50

4.4 主要实验结果

[TABLE:table2 - BLEU scores and training costs]
表 2 展示了 Transformer 在 WMT 2014 翻译任务上的结果：

模型	EN-DE BLEU	EN-FR BLEU	EN-DE FLOPs	EN-FR FLOPs
Transformer (big)	**28.4**	**41.8**	2.3×10^{1}	6.3×10^{1}
GNMT + RL Ensemble	26.3	41.16	1.8×10^{2}	1.1×10^{21}
ConvS2S Ensemble	26.36	41.29	7.7×10^{1}	1.2×10^{21}
MoE	26.03	40.56	2.0×10^{1}	1.2×10^{2}

4.4.1 关键成果

1. **英德翻译**：Transformer (big) 以 **28.4 BLEU** 超越所有先前最佳模型（包括集成模型）**超过 2 个 BLEU 点**
1. **英法翻译**：Transformer (big) 以 **41.8 BLEU** 建立新的单模型最高记录
1. **训练效率**：

- 英德翻译：仅需 **1/4** 其他最佳模型（GNMT）的训练成本
- 英法翻译：仅需 **1/16** 训练成本

4.5 消融实验分析

[TABLE:table3 - Model variations on development set]
表 3 展示了模型各组件的消融实验结果：

4.5.1 注意力头数和维度（行 A）

配置	头数	$d_k = d_v$	BLEU
base	8	64	**25.8**
-	1	512	24.9
-	4	128	**25.5**
-	16	32	25.8
-	32	16	25.4

发现：单头注意力比最佳配置差 0.9 个 BLEU；头数过多或过少都会导致性能下降。

4.5.2 键维度的影响（行 B）

d_k	PPL	BLEU
32	5.01	25.4
64	**4.92**	**25.8**
128	5.16	25.1

发现：减小键维度会损害模型质量，表明确定兼容性的函数比简单点积更复杂可能有益。

4.5.3 模型深度和宽度（行 C、D）

N	d_{ff}	d_{model}	PPL	BLEU
2	2048	512	6.11	23.7
4	2048	512	5.19	25.3
6	**2048**	**512**	**4.88**	**25.5**
8	2048	512	4.66	26.0
6	1024	512	5.12	25.4
6	**4096**	512	**4.75**	**26.2**

发现：更大模型和更大前馈层维度均能提升性能。

4.5.4 Dropout 的影响（行 D）

P_{drop}	PPL	BLEU
0.0	5.77	24.6
0.1	4.92	25.5
0.2	**4.95**	**25.5**

发现：Dropout 对防止过拟合至关重要。

4.5.5 位置编码比较（行 E）

位置编码	BLEU
正弦（本文）	25.8
可学习	25.7

发现：两种方法几乎相同。

4.6 泛化能力：英语 constituency 句法分析

[TABLE:table4 - Parsing results]

为验证 Transformer 的泛化能力，在**英语 constituency 句法分析**任务上进行了实验：

模型	训练设置	WSJ 23 F1
Transformer (4 层)	WSJ only	91.3
Transformer (4 层)	半监督	**92.7**
RNN Grammar	WSJ only	91.7
RNN Grammar	半监督	93.3

发现：

- 仅用 40K 句子的 WSJ 训练集，Transformer 就能超过 BerkeleyParser
- 证明模型具有良好的**跨任务泛化能力**

4.7 注意力可视化分析

论文中的**图 4** 和**图 5** 展示了注意力头的可视化，表明：

1. **不同头学习不同任务**：某些头专门处理长距离依赖，某些处理指代消解
2. **捕捉语言结构**：注意力分布与句子的句法和语义结构相关

---

5. 优缺点评价

5.1 主要优点

5.1.1 卓越的并行化能力 Transformer 完全基于注意力机制，可以**在序列的所有位置并行计算**，训练速度比 RNN 快数个数量级。这使得在合理时间内训练大型模型成为可能。

5.1.2 最优的长距离依赖建模 **最大路径长度为%%INLINEMATH74%%**，常数级复杂度使模型能够轻松学习任意距离位置间的依赖关系，解决了 RNN 的梯度消失/爆炸问题。

5.1.3 突破性的翻译质量

- 在 WMT 2014 英德任务上以 **28.4 BLEU** 超越所有先前模型
- 在 WMT 2014 英法任务上以 **41.8 BLEU** 创下单模型记录
- 证明了注意力机制本身的质量优势

5.1.4 优秀的泛化能力 成功应用于英语句法分析任务，展示了架构的**任务无关性**和**可迁移性**。

5.1.5 可解释性 注意力权重提供了**可视化的注意力分布**，使模型的决策过程更加透明，有助于理解模型学到了什么。

5.2 主要局限性

5.2.1 计算复杂度与序列长度的关系 标准自注意力的复杂度为 $O(n^2 \cdot d)$，当**序列长度%%INLINEMATH76%%非常大**时（如长文档、音频信号），计算成本显著增加。

5.2.2 位置编码的局限 使用固定正弦位置编码，虽然可外推，但可能无法完全捕捉复杂的位置关系。相比可学习的位置嵌入，其表达能力可能受限。

5.2.3 训练稳定性 深层 Transformer（6 层以上）的训练可能需要仔细的超参数调优，如学习率预热、层归一化位置等。

5.3 可能的改进方向

5.3.1 高效注意力机制

- **Sparse Attention**：如 Longformer、BigBird，通过稀疏模式降低复杂度
- **Linformer**：线性复杂度的注意力近似
- **Reformer**：使用局部敏感哈希的注意力

5.3.2 相对位置编码

- **T5 的相对位置编码**：更灵活地处理位置关系
- **Transformer-XL**：段落级别的相对位置编码

5.3.3 轻量化模型

- **DistilBERT**：知识蒸馏压缩
- **MobileBERT**：针对移动端优化

5.3.4 多模态扩展 将 Transformer 扩展到图像（ViT）、音频、视频等模态。

---

6. 个人思考与启发

6.1 核心思想启发

6.1.1 注意力机制的本质力量 这篇论文最深刻的启示是：**注意力机制本身足以完成序列转导任务**。之前的工作将注意力视为 RNN 的辅助工具，而本文证明了注意力可以独立成为模型的核心。

这启示我们：

- 重新审视深度学习中”基础组件”的假设
- 简洁但正确的设计可以超越复杂的工程优化

6.1.2 并行化与序列建模的兼容 Transformer 证明了**并行计算和序列依赖建模并不矛盾**。通过使用全局注意力，模型可以同时：

- 利用 GPU/TPU 的并行计算能力
- 保持对序列中任意位置依赖的建模能力

这对其他序列任务的设计具有重要指导意义。

6.1.3 多头注意力的协同效应

多个注意力头可以学习**不同类型的依赖关系**：

- 某些头专注于局部关系
- 某些头专注于长距离依赖
- 某些头捕捉语法结构
- 某些头捕捉语义关联

这表明通过**模块化设计**，可以让模型的不同组件专注于不同子任务。

6.2 可借鉴的技术

6.2.1 残差连接 + 层归一化

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

这一组合已成为现代深度学习模型的标准组件，对训练深层网络至关重要。

6.2.2 学习率预热策略

$$lrate = d_{\text{model}}^{-0.5} \cdot \min\left(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}\right)$$

预热阶段帮助模型在早期训练中稳定收敛，尤其适用于大型模型。

6.2.3 标签平滑

$$\text{loss} = -\sum_i y_i \cdot \left((1-\epsilon)\log(\hat{y}_i) + \frac{\epsilon}{K}\right)$$

通过引入适度的不确定性，提升模型的泛化能力。

6.3 Follow-up 研究方向

6.3.1 大规模预训练模型

- **BERT**：基于 Transformer 编码器的双向预训练
- **GPT 系列**：基于 Transformer 解码器的语言模型预训练
- **T5**：统一的文本到文本 Transformer

6.3.2 高效 Transformer 变体

- **Linear Attention**：将复杂度降至 $O(n)$
- **Flash Attention**：IO 感知的精确注意力实现
- **Sparse/Linear Transformers**：稀疏模式或线性近似

6.3.3 多模态学习

- **Vision Transformer (ViT)**：图像分类
- **Audio Spectrogram Transformer**：音频处理
- **多模态大模型**：GPT-4、Claude 等

6.3.4 更深入的理论理解

- 注意力机制的理论性质
- Transformer 的表达能力
- 训练动态的理论分析

6.4 对 NLP 研究的长期影响

Transformer 的出现标志着深度学习在 NLP 领域的**范式转变**：

1. **从任务特定到通用**：预训练 + 微调范式
2. **从 RNN 到注意力**：新的模型设计哲学
3. **从工程优化到规模效应**：scaling laws 的发现

---

7. 关键词与核心概念

关键词	英文	简要解释
Transformer	Transformer	完全基于注意力机制的序列转导模型架构
自注意力	Self-Attention	关联同一序列中不同位置以计算表示的注意力机制
多头注意力	Multi-Head Attention	将注意力投影到多个子空间并行计算
缩放点积注意力	Scaled Dot-Product Attention	使用点积和缩放因子的注意力计算方式
编码器-解码器	Encoder-Decoder	序列转导的标准架构：编码输入 + 解码输出
位置编码	Positional Encoding	注入序列位置信息的编码方式
残差连接	Residual Connection	将输入直接加到输出以缓解梯度问题
层归一化	Layer Normalization	归一化技术，稳定训练
束搜索	Beam Search	序列生成中的启发式搜索算法
BLEU	BLEU	机器翻译的标准评估指标
WMT	WMT	机器翻译 Workshop/Conference
词片	Word-piece/BPE	子词分词技术
掩码	Mask	防止关注某些位置的机制
自回归	Auto-regressive	基于先前输出生成下一步输出的特性
标签平滑	Label Smoothing	通过软化标签防止过拟合
Adam 优化器	Adam Optimizer	自适应学习率的优化算法

附录：关键公式汇总

缩放点积注意力：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

多头注意力：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, ..., \text{head}_h)\mathbf{W}^O$$

前馈网络：

$$\text{FFN}(x) = \max(0, x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

位置编码：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

学习率调度：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min\left(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}\right)$$

笔记完成时间：基于 *Transformer* 原论文（*Vaswani et al., 2017, NeurIPS*）